AWS CLI Equivalent

CloudOps 01 monitoring and incident response

This appendix preserves the original guide and adds the complete PowerShell CLI repeat. Use a dedicated sandbox profile and us-east-1. Do not paste credentials into files.

1. Configure and verify the profile

```
aws configure sso
aws sso login --profile YOUR_PROFILE
aws sts get-caller-identity --profile YOUR_PROFILE
```

2. Set variables

```
$Profile = "YOUR_PROFILE"
$Region = "us-east-1"
$FunctionName = "cloudops-01-monitoring"
$LogGroup = "/aws/lambda/$FunctionName"
$RoleName = "$FunctionName-execution-role"
$AlarmName = "$FunctionName-errors"
$AccountId = aws sts get-caller-identity --profile $Profile --query Account --output text
```

3. Create logs and IAM

```
aws logs create-log-group --log-group-name $LogGroup --region $Region --profile $Profile
aws logs put-retention-policy --log-group-name $LogGroup --retention-in-days 1 --region $Region --profile
$Profile

Create trust-policy.json for lambda.amazonaws.com, then run:
aws iam create-role --role-name $RoleName --assume-role-policy-document file://trust-policy.json --profile
$Profile
aws iam put-role-policy --role-name $RoleName --policy-name "$FunctionName-logging" --policy-document
file://logging-policy.json --profile $Profile
Start-Sleep -Seconds 10
```

4. Package and deploy

```
Compress-Archive -Path labs/cloudops/01-monitoring-incidents/src/lambda_function.py -DestinationPath
cloudops-01-monitoring.zip -Force
$RoleArn = aws iam get-role --role-name $RoleName --query Role.Arn --output text --profile $Profile
aws lambda create-function --function-name $FunctionName --runtime python3.12 --handler
lambda_function.lambda_handler --role $RoleArn --zip-file fileb://cloudops-01-monitoring.zip --memory-size 128
--timeout 3 --region $Region --profile $Profile
```

5. Invoke success and failure

```
"{"fail": false}" | Set-Content success.json
"{"fail": true}" | Set-Content failure.json
aws lambda invoke --function-name $FunctionName --payload fileb://success.json --cli-binary-format raw-in-
base64-out success-response.json --region $Region --profile $Profile
aws lambda invoke --function-name $FunctionName --payload fileb://failure.json --cli-binary-format raw-in-
base64-out failure-response.json --region $Region --profile $Profile
# Run the failure invocation three times.
```

6. Create and inspect the alarm

```
aws cloudwatch put-metric-alarm --alarm-name $AlarmName --namespace AWS/Lambda --metric-name Errors --dimensions
Name=FunctionName,Value=$FunctionName --statistic Sum --period 60 --evaluation-periods 1 --datapoints-to-alarm
1 --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold --treat-missing-data notBreaching --region
$Region --profile $Profile

aws cloudwatch describe-alarms --alarm-names $AlarmName --query
"MetricAlarms[0].[AlarmName,StateValue,MetricName,Namespace,Threshold]" --output table --region $Region
--profile $Profile
```

7. Inspect logs and recover

```
aws logs tail $LogGroup --since 30m --format short --region $Region --profile $Profile
# Invoke success.json three times, then wait for one-minute evaluation.
aws cloudwatch describe-alarms --alarm-names $AlarmName --query
"MetricAlarms[0].[AlarmName,StateValue,StateReason]" --output table --region $Region --profile $Profile
```

8. Clean up

```
aws cloudwatch delete-alarms --alarm-names $AlarmName --region $Region --profile $Profile
aws lambda delete-function --function-name $FunctionName --region $Region --profile $Profile
aws logs delete-log-group --log-group-name $LogGroup --region $Region --profile $Profile
aws iam delete-role-policy --role-name $RoleName --policy-name "$FunctionName-logging" --profile $Profile
aws iam delete-role --role-name $RoleName --profile $Profile
```

Terraform section by file

The following section is included in this PDF in the requested file-by-file order. It is sourced from terraform-guide.md.

CloudOps 01 Terraform implementation

This is a separate sandbox implementation of CloudOps-01. It does not use or modify the enterprise `infra/` root. Use a dedicated directory and state for this lab, a nonproduction account, and `us-east-1`.

The files below are shown in the order to create them. Terraform provisions the resources; the AWS CLI or Console still performs the runtime failure and recovery drill.

1. Directory layout

Create this layout from the repository root:

```
labs/cloudops/01-monitoring-incidents/terraform/
  versions.tf
  providers.tf
  variables.tf
  locals.tf
  iam.tf
  logs.tf
  lambda.tf
  cloudwatch.tf
  outputs.tf
  terraform.tfvars.example
```

The Lambda source remains here:

```
labs/cloudops/01-monitoring-incidents/src/lambda_function.py
```

2. `versions.tf`

Create `terraform/versions.tf`:

```
terraform {
  required_version = ">= 1.9.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    archive = {
      source  = "hashicorp/archive"
      version = "~> 2.4"
    }
  }
}
```

3. `providers.tf`

Create `terraform/providers.tf`:

```
provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project     = var.project_name
      Track       = "CloudOps"
      Lab         = "cloudops-01-monitoring"
      Environment = var.environment
      ManagedBy   = "Terraform"
      Owner       = var.owner
    }
  }
}
```

The AWS provider uses the credentials selected by your AWS CLI profile or IAM Identity Center session. Do not add access keys to Terraform files.

4. `variables.tf`

Create `terraform/variables.tf`:

```
variable "aws_region" {
  type        = string
  description = "AWS Region for the lab."
  default     = "us-east-1"
}

variable "project_name" {
  type        = string
  description = "Stable project name."
  default     = "content-failures-api"
}

variable "environment" {
  type        = string
  description = "Sandbox environment label."
  default     = "sandbox"

  validation {
    condition     = contains(["sandbox", "test"], var.environment)
    error_message = "Use sandbox or test for this exercise."
  }
}

variable "owner" {
  type        = string
  description = "Person responsible for cleanup."
}

variable "function_name" {
  type        = string
  description = "Lambda function name."
  default     = "cloudops-01-monitoring"
}
```

5. `locals.tf`

Create `terraform/locals.tf`:

```
locals {
  log_group_name = "/aws/lambda/${var.function_name}"
  role_name      = "${var.function_name}-execution-role"
  alarm_name     = "${var.function_name}-errors"
  source_file    = "${path.module}/../src/lambda_function.py"
}
```

6. `iam.tf`

Create `terraform/iam.tf`:

```
resource "aws_iam_role" "lambda" {
  name = local.role_name

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Service = "lambda.amazonaws.com"
      }
      Action = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy" "lambda_logs" {
  name = "${var.function_name}-logging"
  role = aws_iam_role.lambda.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = [
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ]
      Resource = "${aws_cloudwatch_log_group.lambda.arn}:*"
    }]
  })
}
```

The policy intentionally omits `logs:CreateLogGroup` because Terraform creates the group first.

7. `logs.tf`

Create `terraform/logs.tf`:

```
resource "aws_cloudwatch_log_group" "lambda" {
  name              = local.log_group_name
  retention_in_days = 1
}
```

8. `lambda.tf`

Create `terraform/lambda.tf`:

```
data "archive_file" "lambda" {
  type        = "zip"
  source_file = local.source_file
  output_path = "${path.module}/lambda.zip"
}

resource "aws_lambda_function" "monitoring" {
  function_name    = var.function_name
  role             = aws_iam_role.lambda.arn
  runtime          = "python3.12"
  handler          = "lambda_function.lambda_handler"
  filename         = data.archive_file.lambda.output_path
  source_code_hash = data.archive_file.lambda.output_base64sha256
  memory_size     = 128
  timeout         = 3

  depends_on = [aws_cloudwatch_log_group.lambda]
}
```

If the AWS Console shows a different currently supported Python runtime, use that runtime consistently in the plan and record the choice.

9. `cloudwatch.tf`

Create `terraform/cloudwatch.tf`:

```
resource "aws_cloudwatch_metric_alarm" "errors" {
  alarm_name          = local.alarm_name
  alarm_description   = "Lambda errors for the CloudOps-01 monitoring lab."
  namespace           = "AWS/Lambda"
  metric_name         = "Errors"
  dimensions          = { FunctionName = aws_lambda_function.monitoring.function_name }
  statistic           = "Sum"
  period              = 60
  evaluation_periods  = 1
  datapoints_to_alarm = 1
  threshold           = 1
  comparison_operator = "GreaterThanOrEqualToThreshold"
  treat_missing_data  = "notBreaching"
}
```

The metric name is `Errors`. The alarm name is `cloudops-01-monitoring-errors`; they are different values.

10. `outputs.tf`

Create `terraform/outputs.tf`:

```
output "function_name" {
  value = aws_lambda_function.monitoring.function_name
}

output "function_arn" {
  value = aws_lambda_function.monitoring.arn
}

output "log_group_name" {
  value = aws_cloudwatch_log_group.lambda.name
}

output "alarm_name" {
  value = aws_cloudwatch_metric_alarm.errors.alarm_name
}

output "execution_role_name" {
  value = aws_iam_role.lambda.name
}
```

11. `terraform.tfvars.example`

Create `terraform/terraform.tfvars.example`:

```
aws_region   = "us-east-1"
project_name = "content-failures-api"
environment  = "sandbox"
owner        = "YOUR_NAME"
```

Copy it to a local `terraform.tfvars`, replace `YOUR_NAME`, and do not commit the local file if it contains personal or account-specific values.

12. Initialize and validate

From `labs/cloudops/01-monitoring-incidents/terraform`:

```
Copy-Item terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars and replace YOUR_NAME.

terraform init
terraform fmt -check
terraform validate
terraform plan -out cloudops-01.tfplan
```

Review the plan. Verify the account, Region, names, IAM policy, log retention, alarm metric, and resource count before applying.

13. Apply and perform the runtime drill

Apply only after reviewing the sandbox plan:

```
terraform apply cloudops-01.tfplan
terraform output
```

Use the [AWS CLI equivalent](cli-equivalent.md) to invoke `fail=true`, confirm `ALARM`, invoke `fail=false`, and confirm `OK`. Terraform does not replace runtime failure testing.

14. Destroy and verify cleanup

```
terraform plan -destroy -out cloudops-01-destroy.tfplan
terraform apply cloudops-01-destroy.tfplan
```

Confirm the plan contains only this lab's resources. Verify that the Lambda, log group, alarm, role, policy, and generated `lambda.zip` are removed as intended. Never commit `.terraform`, `terraform.tfstate`, `terraform.tfstate.backup`, `terraform.tfvars`, credentials, or secrets.

Important: the CloudWatch metric name is Errors. The alarm name is cloudops-01-monitoring-errors. The original guide pages, including their images, are preserved unchanged before this appendix.